Київський національний університет імені Тараса Шевченка
Факультет комп’ютерних наук та кібернетики

ЗВІТ ДО ЛАБОРАТОРНОЇ РОБОТИ 1:

Паралельна обробка гістограм зображень у системі ПАРКС

Вирівнювання гістограми на PARCS-Python у Google Cloud (тема 5.2)

З дисципліни «Хмарні обчислення»

Виконала студентка 4-го курсу
групи ТТП-42
Рябіченко Євгенія Дмитрівна

Київ-2026

ЗМІСТ

1. Тема, мета та завдання 3
2. Опис алгоритму 3
2.1. Послідовне вирівнювання гістограми 3
2.2. Паралельна схема map → reduce → map 4
3. Програмна реалізація 5
3.1. Як працює PARCS-Python 5
3.2. Структура проєкту 5
3.3. Ключові фрагменти коду 5
4. Розгортання в Google Cloud 7
4.1. Гайд з налаштування (мінімальний набір команд) 8
5. Проблеми збірки та розгортання і їх вирішення 9
6. Результати експериментів 10
6.1. Аналіз результатів 12
7. Висновки 12
8. Посилання 13

1. Тема, мета та завдання

Тема: паралельна обробка гістограм зображень: підрахунок і вирівнювання гістограми для покращення контрастності (тема 5.2 зі списку тем курсу «Розподілене та паралельне програмування»).

Мета: навчитися розгортати систему паралельних обчислень ПАРКС у хмарному середовищі, реалізувати паралельний алгоритм для неї та дослідити, як час виконання залежить від кількості воркерів.

Завдання:

- обрати задачу паралельного програмування та платформу ПАРКС (обрано PARCS-Python);
- розгорнути систему ПАРКС у хмарі: master і 4 worker-вузли в Google Cloud Compute Engine;
- реалізувати послідовний і паралельний алгоритми вирівнювання гістограми;
- провести тестування для 1, 2, 3 і 4 воркерів, побудувати таблицю і графік результатів;
- описати проблеми збірки та їх вирішення, скласти гайд з налаштування хмари;
- завантажити код на GitHub.

Інструментарій: Python 2.7 (середовище образу PARCS-Python) для solver-a, Python 3 з numpy, Pillow і matplotlib для підготовки даних та графіків, Google Cloud SDK (gcloud), Docker.

2. Опис алгоритму

2.1. Послідовне вирівнювання гістограми

Гістограма зображення h[v] — кількість пікселів з яскравістю v, де v = 0…255. Якщо значення зосереджені у вузькому діапазоні (наприклад 64–126), зображення виглядає «блідим». Вирівнювання гістограми перерозподіляє рівні яскравості так, щоб вони рівномірно займали весь діапазон 0–255.

Алгоритм для зображення з T пікселів:

- обчислити гістограму h[v] — один прохід по всіх пікселях;
- обчислити кумулятивну функцію розподілу cdf[v] = h[0] + h[1] + … + h[v];

- побудувати таблицю перетворення lut[v] = round((cdf[v] − cdf_min) / (T − cdf_min) · 255), де cdf_min — перше ненульове значення cdf;
- замінити кожен піксель p на lut[p] — другий прохід по всіх пікселях.

Для кольорових зображень (RGB) кожен піксель переводиться у простір YCbCr, вирівнюється лише яскравість Y, а кольоровість Cb і Cr зберігається. Якщо вирівнювати канали R, G, B окремо, змінюється баланс кольорів і зображення отримує паразитні відтінки.

Складність послідовного алгоритму — O(T + 256): два проходи по пікселях і побудова таблиці з 256 елементів.

2.2. Паралельна схема map → reduce → map

Ключова властивість, яка дозволяє розпаралелити задачу: гістограма адитивна. Якщо розбити зображення на частини і порахувати гістограму кожної частини окремо, то сума цих гістограм дорівнює гістограмі всього зображення. Тому схема така:

Таблиця 1. Етапи паралельного алгоритму

Етап	Хто виконує	Що передається	Складність
Розбиття на N горизонтальних смуг	master	смуга пікселів кожному воркеру	O(T)
MAP-1: локальна гістограма смуги	кожен воркер	256 чисел назад на master	O(T/N)
REDUCE: сума гістограм, CDF, LUT	master	—	O(256·N)
MAP-2: застосування LUT до смуги	кожен воркер	LUT (256 чисел) → смуга назад	O(T/N)
Склеювання смуг, запис результату	master	—	O(T)

Смуги горизонтальні, бо рядки зображення лежать у пам’яті послідовно — смуга вирізається одним зрізом масиву. Обсяг роботи на кожен піксель однаковий, тому рівні за висотою смуги дають рівномірне навантаження на воркери. Висоти смуг відрізняються максимум на один рядок.

Результат не залежить від кількості воркерів: таблиця LUT будується з глобальної гістограми, тож кожен піксель отримує те саме значення, що й у послідовному алгоритмі. Це перевірено побайтовим порівнянням вихідних файлів для N = 1…4 і порівнянням з еталонною реалізацією на numpy.

3. Програмна реалізація

3.1. Як працює PARCS-Python

PARCS-Python складається з master-вузла з веб-інтерфейсом (порт 8080) і worker-вузлів, які реєструються на master за його внутрішньою IP-адресою. Користувач через сторінку Add Job завантажує файл розв'язку (Solution File) і вхідний файл (Input File). Master надсилає файл розв'язку кожному увімкненому воркеру; воркер запускає Pyro4-сервер і повертає його URI. Далі master створює об'єкт класу `Solver` зі списком проксі-об'єктів воркерів і викликає метод `solve()`. Методи, позначені декоратором `@expose`, виконуються на воркерах як віддалені виклики.

Образ `hummer12007/parcs-node` містить Python 2.7.13 і Pyro4 4.74, numpy і Pillow в ньому немає. Тому solver написано на чистому Python, сумісно з версіями 2.7 і 3.x, а зображення передається у бінарному форматі PGM (P5, відтінки сірого) або PPM (P6, RGB).

3.2. Структура проєкту

Таблиця 2. Структура репозиторію

Файл	Призначення
solver/histeq_solver.py	Solution File для PARCS: код master (solve) і воркерів (@expose)
tools/make_input.py	Фото або синтетичне зображення → низькоконтрастний PGM/PPM
tools/local_run.py	Прогін solver-а без кластера (послідовна перевірка)
tools/local_cluster.sh	Локальний PARCS-кластер у Docker (master + N воркерів)
tools/benchmark.py	Автоматичний прогін для N = 1…4 через HTTP API PARCS
tools/plot_results.py	Таблиця і графік з CSV-результатів
gcp/startup-master.sh, startup-worker.sh	Startup-скрипти VM у Google Cloud
docs/GCP_RUN_GUIDE.md	Покроковий гайд з розгортання

3.3. Ключові фрагменти коду

Метод `solve()` виконується на master і реалізує всю схему map → reduce → map:

```
def solve(self):
    magic, width, height, pixels = self.read_input()
    channels = 3 if magic == 'P6' else 1
    strips = self.split(pixels, width * channels, height, n)
    wire = [_to_wire(bytes(s)) for s in strips]
    # MAP-1: local histograms
    hists = self.parallel_call('local_histogram', [(w, channels) for w in wire])
    # REDUCE: global histogram -> CDF -> LUT
    lut = Solver.build_lut(Solver.merge_histograms(hists))
    # MAP-2: apply LUT
    results = self.parallel_call('apply_lut', [(w, channels, lut) for w in wire])
    ...
    self.write_output(magic, width, height, out, stats)
```

Виклики воркерів робляться одночасно: кожен виклик — в окремому потоці, master чекає завершення всіх. Якби виклики йшли в циклі один за одним, кожен наступний воркер починав би роботу лише після попереднього, і паралелізму не було б.

```
def parallel_call(self, method, args_list):
    results = [None] * len(args_list)
    def run(i):
        proxy = self.workers[i]
        results[i] = _value(getattr(proxy, method)(*args_list[i]))
    threads = [threading.Thread(target=run, args=(i,)) for i in
range(len(args_list))]
    for t in threads: t.start()
    for t in threads: t.join()
    return results
```

Методи воркерів — підрахунок гістограми смуги і побудова таблиці перетворення:

```
@staticmethod
@expose
def local_histogram(strip_wire, channels):
    data = _from_wire(strip_wire)
    hist = [0] * 256
    if channels == 1:
        for p in data:
            hist[p] += 1
    else:
        for i in range(0, len(data), 3):
            hist[Solver.luma(data[i], data[i + 1], data[i + 2])] += 1
    return hist


@staticmethod
def build_lut(hist):
    cdf, acc = [], 0
    for c in hist:
        acc += c; cdf.append(acc)
    total = cdf[-1]
    cdf_min = next(c for c in cdf if c > 0)
    scale = 255.0 / (total - cdf_min)
    return [max(0, min(255, int(round((c - cdf_min) * scale)))) for c in cdf]
```

Смуга передається як один base64-рядок: серіалізатор Pyro4 (serpent) передає великий рядок у рази швидше, ніж список з мільйонів чисел. Час

кожної фази solver записує в заголовок вихідного файлу коментарями `# t_total=…`, бо веб-інтерфейс PARCS показує тривалість задачі лише в цілих секундах.

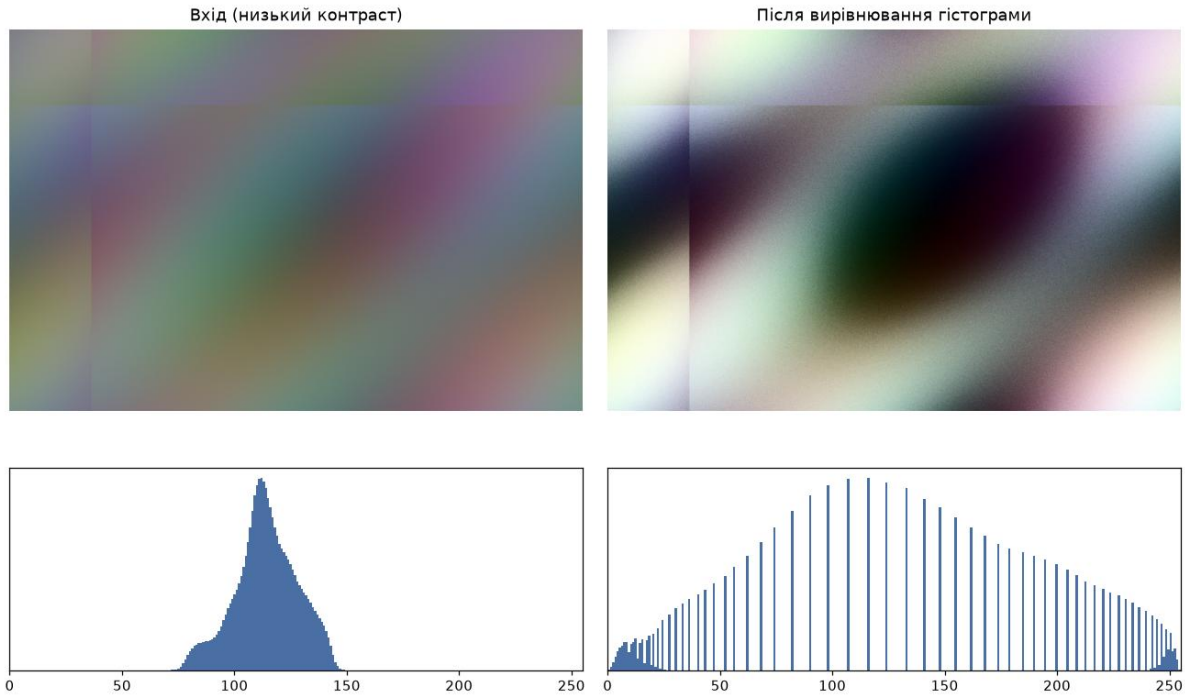


Рис. 1. Вхідне зображення (низький контраст) і результат вирівнювання з гістограмами яскравості

4. Розгортання в Google Cloud

Кластер розгорнуто в Google Cloud Compute Engine, регіон europe-central2 (Варшава): одна VM master і чотири VM worker1–worker4 типу e2-medium (2 vCPU, 4 ГБ RAM, Intel Xeon 2.20 GHz). Кожна VM працює на Container-Optimized OS і запускає Docker-контейнер `hummer12007/parcs-node` через startup-скрипт: master — з аргументом `PARCS_ARGS=master`, воркер — з `PARCS_ARGS="worker <внутрішня IP master>"`.

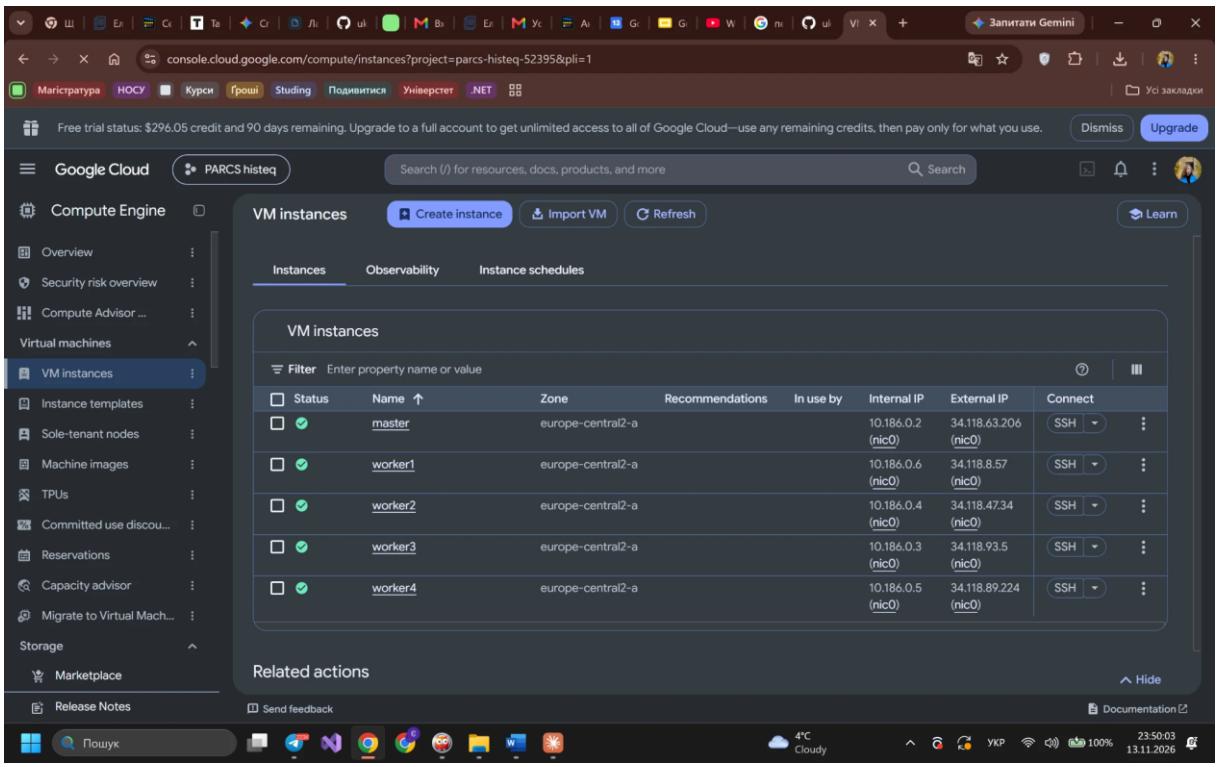


Рис. 2. Google Cloud Console: master і worker1–worker4 у стані RUNNING

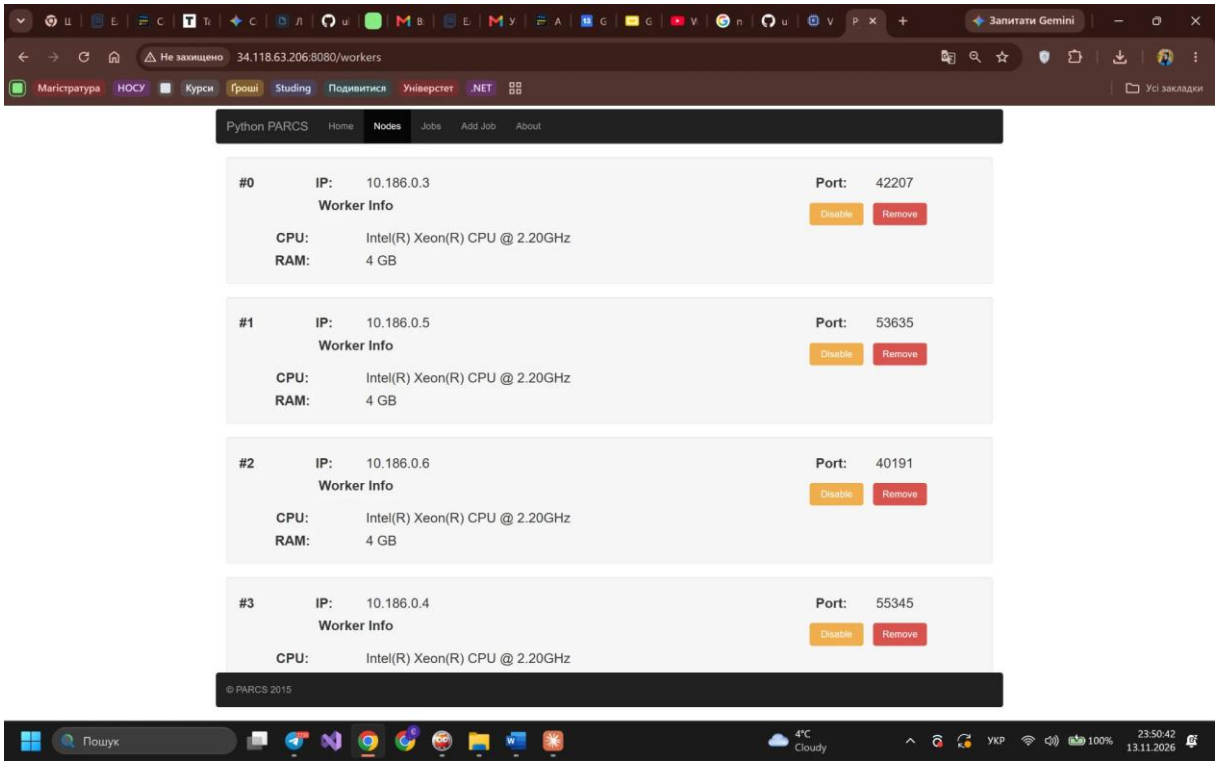


Рис. 3. Веб-інтерфейс PARCS, сторінка Nodes: 4 зареєстровані воркери

4.1. Гайд з налаштування (мінімальний набір команд)

Команди для PowerShell; повна версія з поясненнями — у файлі `docs/GCP_RUN_GUIDE.md` репозиторію.

```
# 1. Проєкт, API, зона (білінг: Start free trial у консолі, прив'язати до проєкту)
gcloud auth login
gcloud projects create parcs-histeq-XXXXX
gcloud config set project parcs-histeq-XXXXX
gcloud services enable compute.googleapis.com
gcloud config set compute/zone europe-central2-a

# 2. Фаєрвол: порт 8080 лише для своєї IP (внутрішній трафік дозволено за
замовчуванням)
gcloud compute firewall-rules create parcs-web --allow=tcp:8080 --source-
ranges=<MY_IP>/32

# 3. Master
gcloud compute instances create master --machine-type=e2-medium `
  --image-family=cos-stable --image-project=cos-cloud `
  --metadata-from-file=startup-script=gcp/startup-master.sh

# 4. Workers (MASTER_IP - внутрішня IP master)
gcloud compute instances create worker1 worker2 worker3 worker4 --machine-type=e2-
medium `
  --image-family=cos-stable --image-project=cos-cloud `
  --metadata=parcs-master-ip=<MASTER_IP> `
  --metadata-from-file=startup-script=gcp/startup-worker.sh

# 5. Запуск: http://<MASTER_EXTERNAL_IP>:8080 -> Add Job, або автоматично:
python tools/benchmark.py --master http://<MASTER_EXTERNAL_IP>:8080 `
  --input data/input_rgb.txt --workers 1 2 3 4 --repeat 3 --csv results/gcp.csv

# 6. Прибрати за собою
gcloud compute instances delete master worker1 worker2 worker3 worker4 --quiet
```

Startup-скрипт воркера (фрагмент): IP master читається з метаданих VM, а в iptables відкривається трафік із внутрішньої мережі.

```
MASTER_IP=$(curl -s -H "Metadata-Flavor: Google" \
  "http://metadata.google.internal/computeMetadata/v1/instance/attributes/parcs-
master-ip")
iptables -A INPUT -s 10.128.0.0/9 -j ACCEPT
docker run -d --name parcs --restart=always --net=host \
  -e PARCS_ARGS="worker ${MASTER_IP}" hummer12007/parcs-node
```

5. Проблеми збірки та розгортання і їх вирішення

Таблиця 3. Проблеми та рішення

Проблема	Як проявилась	Рішення
PARCS-Python написаний під Python 2.7	Код PARCS використовує `Pyro4.async(...)`; у Python ≥ 3.7 `async` — зарезервоване слово	Готовий образ з Docker Hub; solver сумісний з Python 2.7 і 3.x (без f-strings)
Образ не збирається з вихідників	apt-get update → 404 (Debian stretch переїхав в архів); pip у Python 2.7 → CERTIFICATE_VERIFY_FAILED	Не перезбирати, використовувати готовий образ hummer12007/parcs-node
В образі немає numpy і Pillow	import numpy у solver → помилка задачі	Формат PGM/PPM і розбір заголовка на чистому Python

Проблема	Як проявилась	Рішення
Тривалість задачі у веб-інтерфейсі — цілі секунди	Для малих зображень усі прогони «0:0:1»	Власні таймери фаз, запис у заголовок вихідного файлу
Команда create-with-container з методички вимкнена Google	ERROR: … container startup agent is discontinued	VM на Container-Optimized OS + startup-скрипт з docker run --net=host
На COS-VM веб-інтерфейс недоступний	Порт 8080 не відповідає, хоча контейнер працює	У COS свій iptables (лише SSH) → відкрити 8080 і 10.128.0.0/9 у startup-скрипті
gcloud auth login падає	SSLError CERTIFICATE_VERIFY_FAILED (Avast перехоплює HTTPS)	Корінь Avast додано в CA-бандл: gcloud config set core/custom_ca_certs_file
Docker Desktop не стартує (локальні тести)	removing stale socket … cannot be accessed	wsl --shutdown, перейменувати %LOCALAPPDATA%\Docker\run, перезапуск

6. Результати експериментів

Вхідні дані — синтетичне низькоконтрастне RGB-зображення 3000×2000 пікселів (18 МБ), створене скриптом `tools/make_input.py`. Кількість воркерів змінювалась кнопками Enable/Disable на сторінці Nodes. Для кожного N виконано 3 прогони, у таблиці — середнє значення. Час виміряно всередині solver-а (без завантаження файлу через браузер).

Таблиця 4. Час виконання в Google Cloud (master + 4 × e2-medium)

Воркери	MAP-1 (гістограми), с	REDUCE, с	MAP-2 (LUT), с	Загальний час, с	Прискорення	Ефективність
1	6.36	0.000	18.05	**24.56**	1.00×	100%
2	3.30	0.000	9.34	**12.74**	1.93×	96%
3	2.20	0.000	6.49	**8.76**	2.80×	93%
4	1.82	0.000	4.94	**6.83**	3.59×	90%

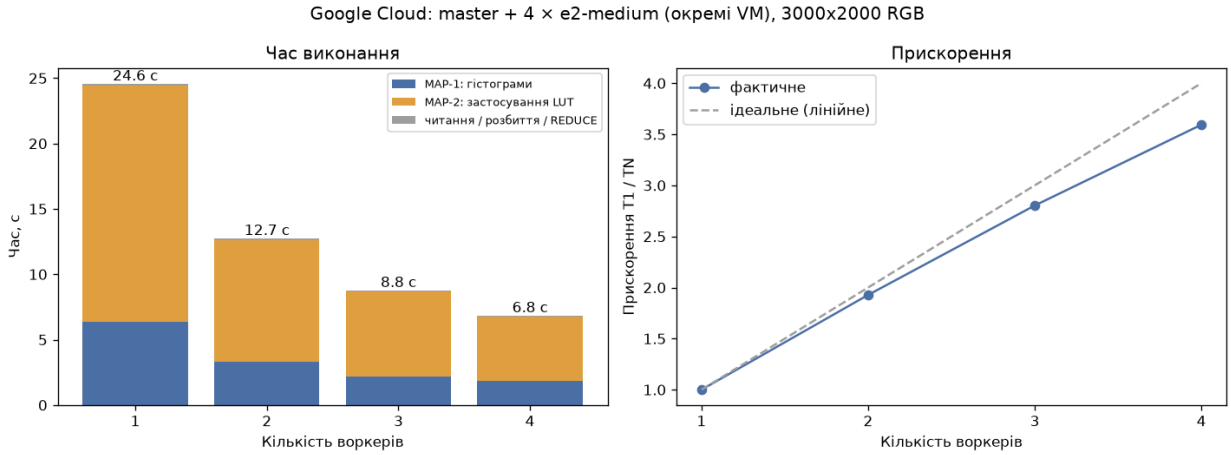


Рис. 4. Залежність часу виконання (ліворуч) і прискорення (праворуч) від кількості воркерів, Google Cloud

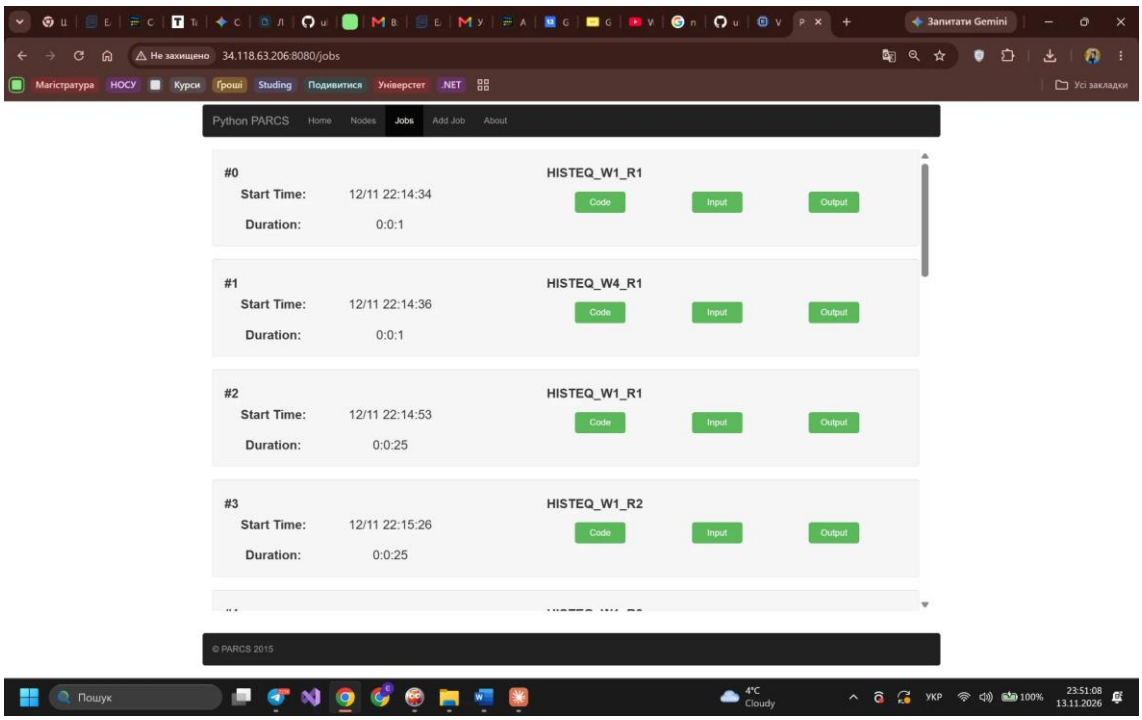


Рис. 5. Сторінка Jobs веб-інтерфейсу PARCS з виконаними задачами

Для порівняння той самий кластер запускався локально в Docker на одному ноутбуці (Intel i7-11370H, 4 ядра):

Таблиця 5. Час виконання на локальному Docker-кластері

Воркери	Загальний час, с	Прискорення	Ефективність
1	13.24	1.00×	100%
2	9.31	1.42×	71%
3	6.34	2.09×	70%
4	5.17	2.56×	64%

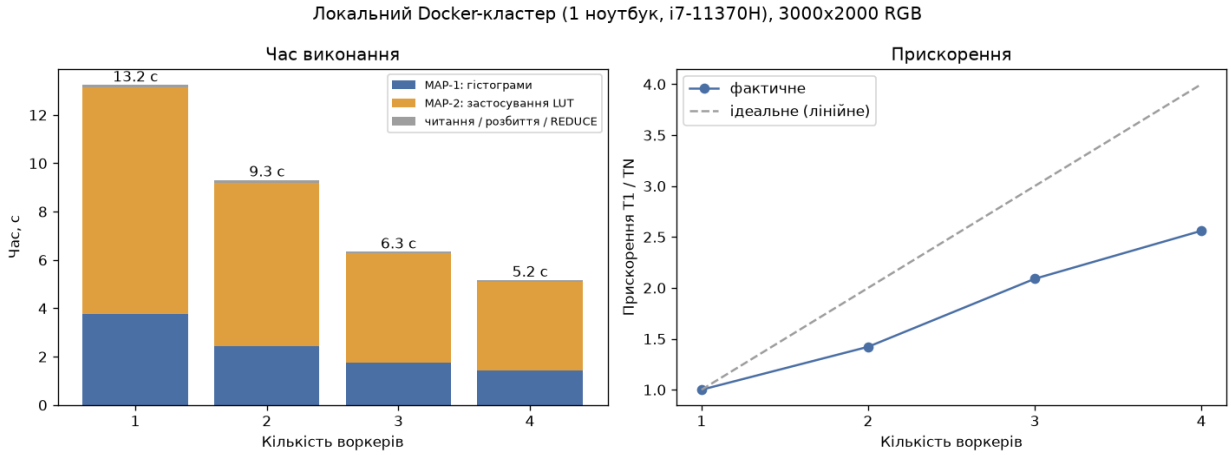


Рис. 6. Результати на локальному Docker-кластері

6.1. Аналіз результатів

- **Майже лінійне прискорення в хмарі.** Час зменшився з 24.6 до 6.8 с (3.59× на 4 воркерах, ефективність 90%). Фаза REDUCE (256 чисел) займає мікросекунди.
- **Чому не рівно 4×.** За законом Амдала прискорення обмежує послідовна частина на master: читання файлу, розбиття, base64-кодування і склеювання смуг, а також передача смуг мережею.
- **Локально гірше.** На одному ноутбуці всі контейнери ділять 4 ядра одного процесора, тому прискорення лише 2.56×. У хмарі кожен воркер має власну VM.
- **Співвідношення обчислень і передачі.** Для сірого зображення 4000×3000 на піксель припадає лише одна операція, і передача даних з'їдає виграш (2.6 с → 1.7 с на 4 воркерах). Для RGB кожен піксель проходить перетворення YCbCr, тож обчислень більше і масштабування добре помітне.
- **Коректність.** Вихідні зображення для N = 1…4 побайтово однакові.

7. Висновки

У роботі реалізовано паралельне вирівнювання гістограми зображення за схемою map → reduce → map на платформі PARCS-Python і розгорнуто кластер з master і чотирьох воркерів у Google Cloud. Адитивність гістограми дозволяє рахувати її частинами без втрати точності, тому результат не залежить від кількості воркерів.

Експерименти показали майже лінійне прискорення: 3.59× на чотирьох воркерах з ефективністю 90%. Обмеження прискорення пояснюється послідовною частиною роботи master і мережевою передачею. Під час роботи довелося розв’язати низку проблем із застарілим програмним забезпеченням: PARCS-Python працює лише на Python 2.7, а описаний у методичці спосіб створення VM з контейнером Google вже вимкнув, тому його замінено на Container-Optimized OS зі startup-скриптами.

8. Посилання

- Код проєкту: https://github.com/ukurere/parcs-histogram-equalization
- PARCS-Python: https://github.com/Hummer12007/parcs-python
- Методичні рекомендації: https://mi.csc.knu.ua/wp-content/uploads/2025/09/Task_1_2_Cloud_Computing_0925.pdf